A Syntax for Connection Proofs

Jens Otten[*] and Sean B. Holden[†]

18/12/23

Abstract

Providing a proof certificate is one of the most important features of (fully automated) theorem proving systems. For theorem provers based on, e.g, resolution or superposition calculi, a syntax for writing solutions is well documented and used. Even though attempts have been made to specify a syntax for connection calculi, there is currently no such syntax in common use. This paper provides an overview of the proof syntax used by existing connection provers and proposes an improved syntax for connection proofs. Due to the general approach, the syntax can be extended to also represent connection proofs in some non-classical logics.

Keywords: Automated Reasoning, logic, connection calculus, connection proofs, syntax, TPTP, proof certificate

1 Introduction

One of the most important features of automated theorem proving (ATP) systems is the output of a proof certificate, which contains a detailed description of the proof and that can be verified by an external (independent) tool. Representing these proofs in a standardized format increases the interoperability between ATP systems, ATP tools, and application software. Such ATP tools can, for example, be used to represent found proofs in a more readable form.

The writing of derivations in resolution calculi using the TPTP/TSTP syntax is well documented and specified [20]. For many other calculi, such as connection or sequent calculi, such a common specification does not exist. Derivations in these calculi differ significantly from derivations in the resolution calculus. Whereas the leaves of a proof in the (formal) connection calculus consist of the axioms of the *calculus*, the leaves of a (TSTP) derivation in the resolution calculus consist of the axiom formulae of the given *problem*.

This paper proposes a syntax for connection proofs. It should not be seen as a final specification, but as a foundation to initiate discussions within the community in order to finalize such a specification. We start with a description of the requirements for a standardized syntax (Section 2), before presenting details of existing syntaxes for proofs, including the well-known TPTP syntax and the syntaxes used by the connection provers leanCoP and Connect++ (Section 3). Afterwards, we specify our proposed syntax for connection proofs (Section 4), before concluding with a summary and outlook (Section 5).

[*]Department of Informatics, University of Oslo, Norway
[†]Department of Computer Science and Technology, University of Cambridge, United Kingdom

2 Preliminaries

We describe requirements for a proof syntax and introduce the formal connection calculus.

2.1 Requirements for a Proof Syntax

There are certain requirements that a proof syntax should fulfill in order be widely accepted by the community. These are:

- *Simple/lean*: The syntax should be as concise and simple as possible allowing a natural and straightforward representation of proofs. A proof syntax that is appropriate for one calculus might not be suitable to represent proofs in another calculus.
- *Human-readable*: Even though one of the main purposes of a standardized proof syntax is the interoperability between ATP systems, ATP tools and application software, a proof in such a syntax should be readable by a human.
- *Based on established formats*: The proof syntax should consider the syntaxes of previous proof formats, whenever these can be used in a straightforward way and do not make the proof representation more complicated than necessary (see first requirement).
- *Well documented and specified*: The proof syntax should be described in a concise and unambiguous way.
- *Extendable*: The proof syntax should be kept general enough to be extendable to similar or related proof calculi, for example for non-classical logics.
- *Efficiently verifiable*: It should be possible to verify the correctness of a given "proof" in at most polynomial time. This excludes, for example, "proofs" consisting only of a set of unsatisfiable clauses, for which finding a *real* proof is NP-complete.

In general, a *proof* of (the validity of) a first-order formula F is a proof of F in a (correct) proof calculus. We will focus on the clausal connection calculus.

2.2 The Connection Calculus

Connection calculi, such as the connection method [2], the connection tableau calculus [8] and the model elimination calculus [9], are established proof search calculi. We use the standard language of *classical first-order logic*. A *(first-order) formula* (denoted by F) is built up from atomic formulae, denoted by A, the connectives $\neg$, $\wedge$, $\vee$, $\Rightarrow$, and the first-order quantifiers $\forall$ and $\exists$. A *literal* L has the form A or $\neg A$.

In the *clausal connection calculus* [2, 14] a formula is represented as a matrix. The *(classical) matrix* $M(F)$ of a formula F is its representation as a set/list of clauses, where each clause is a set/list of literals. It is the representation of F in *disjunctive normal form*. Skolemization of the Eigenvariables is done in the usual way. In the *graphical representation* of a matrix, its clauses are arranged horizontally, while the literals of each clause are arranged vertically.

In contrast to sequent calculi [5] and (standard) tableau calculi, the proof search in connection calculi is guided by connections. A *connection* is a set $\{A_1, \neg A_2\}$ of literals with the same atomic formula, but different *polarities*. A *term substitution* σ assigns terms to variables. A connection $\{A_1, \neg A_2\}$ is *σ-complementary* iff $\sigma(A_1) = \sigma(A_2)$.

Axiom (A) $\dfrac{}{\{\}, M, Path}$ $C_1 \in M$

Start (S) $\dfrac{C_2, M, \{\}}{\varepsilon,\ M,\ \varepsilon}$ C_2 is a copy of $C_1 \in M$

Reduction (R) $\dfrac{C, M, Path \cup \{L_2\}}{C \cup \{L_1\}, M, Path \cup \{L_2\}}$ $\{L_1, L_2\}$ is σ-complementary

Extension (E) $\dfrac{C_2 \backslash \{L_2\}, M, Path \cup \{L_1\} \quad C, M, Path}{C \cup \{L_1\}, M, Path}$

C_2 is a copy of $C_1 \in M$, $L_2 \in C_2$, $\{L_1, L_2\}$ is σ-complementary

Figure 1: The clausal connection calculus for classical logic.

The connection calculus has the three inference rules *Start*, *Reduction*, *Extension*, and one *Axiom*. It is shown in Figure 1; for details see [2, 8, 14, 11]. The words of the calculus are tuples "$C, M,$ *Path*", where M is a matrix, C is a (subgoal) clause or ε and (the active) *Path* is a set of literals or ε. A *copy of a clause* C is made by renaming all variables in C. The *rigid* term substitution σ is calculated by using one of the well-known *term unification* algorithms whenever a connection is identified. The connection calculus is sound and complete [2].

Definition 1 (Connection Proof) *A* connection proof *of M is a derivation of $\varepsilon, M, \varepsilon$ with a term substitution σ in the connection calculus, in which all leaves are closed by axioms.*

Example 1 *A slightly simplified version[1] of Pelletier's problem 24 [17], called Pelletier 24a, consists of the following four axioms[2] and one conjecture formulae.[3]*

$\exists x(Px \wedge Rx)$	*Conjecture*	$\{Px, Rx\}$	(1)
$\neg(\exists x(Sx \wedge Qx))$	*Axiom 1*	$\{Sy, Qy\}$	(2)
$\exists x Px$	*Axiom 3a*	$\{\neg Pa\}$	(3)
$\forall x(Qx \Rightarrow Sx)$	*Axiom 4a*	$\{Qv, \neg Sv\}$	(4)
$\forall(Px \Rightarrow (Qx \vee Rx))$	*Axiom 2*	$\{Pz, \neg Qz, \neg Rz\}$	(5)

Translating these formulae into clausal form (shown in the third column[4]) results in the matrix $M_a = \{\{Px, Rx\}, \{Sy, Qy\}, \{\neg Pa\}\{Qv, \neg Sv\}, \{Pz, \neg Qz, \neg Rz\}\}$. The connection proof for M_a is shown below. Its graphical matrix representation is shown in Figure 2. The proof steps are labeled with the number of the used clause C_1 and the term substitution applied to it.

[1] The original axioms 3 and 4 are $\neg(\exists x Px) \Rightarrow \exists y Qy$ and $\forall x((Qx \vee Rx) \Rightarrow Sx)$, respectively.

[2] Note the difference between the axioms of the problem (the input formula) and the axiom of the calculus.

[3] Axiom 2 is moved to the end of the set of axioms in order to comply with the proofs in Section 3.

[4] A conjecture or axiom might result in more than one clause, which are marked with distinct numbers.

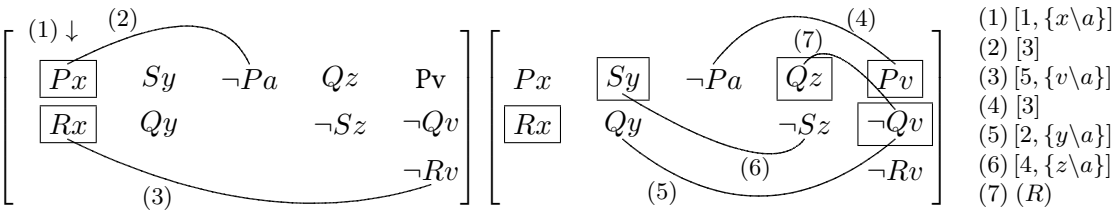


Figure 2: A proof for M_a using the graphical matrix representation.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\{\}, M_a, \{Rx, \neg Qv, Sy\}}\,A}{\{Qz\}, M_a, \{Rx, \neg Qv, Sy\}}\,R \quad \dfrac{}{\{\}, M_a, \{Rx, \neg Qv\}}\,A}{\{Sy\}, M_a, \{Rx, \neg Qv\}}\,E\,(4, \{z\backslash a\}) \quad \dfrac{}{\{\}, M_a, \{Rx\}}\,A}{\{\neg Qv\}, M_a, \{Rx\}}\,E\,(2, \{y\backslash a\})$$

$$\dfrac{\dfrac{}{\{\}, M_a, \{Rx, Pv\}}\,A \qquad \dfrac{\dfrac{}{\{\}, M_a, \{Px\}}\,A \qquad \dfrac{}{\{\}, M_a, \{\}}\,A}{\{Rx\}, M_a, \{\}}\,E\,(5, \{v\backslash a\})}{\{Pv, \neg Qv\}, M_a, \{Rx\}}\,E\,(3)}{\{Px, Rx\}, M_a, \{\}}\,E\,(3)$$

$$\dfrac{\{Px, Rx\}, M_a, \{\}}{\varepsilon, M_a, \varepsilon}\,S\,(1, \{x\backslash a\})$$

3 Existing Syntaxes for Connection Proofs

We present the syntax used for the TPTP library and by the provers leanCoP and Connect++.

3.1 The TPTP/TSTP Syntax

The TPTP and TSTP languages are suitable for representing problems and derivations in first-order and higher-order logic [19, 20]. The top level building blocks are of the following form:

language(*name* , *role* , *formula* , *source* , *useful_info*) .

language is, for example, `fof` for *first-order form* or `cnf` for (conjunctive *or* disjunctive) *clause normal form*. Each formula has a unique *name*. *role* is a label such as `axiom` or `conjecture`. The *source* describes where the formula came from, for example an input file, and *useful_info* is a list of useful information. The last two fields are optional.

Example 2 *Pelletier's original problem 24 is included as problem* `SYN054+1` *in the TPTP library. The representation of its simplified version 24a from Example 1 in TPTP syntax is given in Figure 3.*

A *derivation* or *proof* written in the TPTP language is a list of annotated formulae, as for problems. For derivations/proofs the *source* has one of the forms

```
%----------------------------------------------------------------------
fof(pel24,conjecture, ( ? [X] : ( bigp(X) & bigr(X) ) )).
fof(pel241,axiom, (  ( ? [X] : ( bigs(X) & bigq(X) ) ) )).
fof(pel242,axiom, ( ! [X] : ( bigp(X) =¿ ( bigq(X) — bigr(X) ) ) )).
fof(pel243a,axiom, ( ? [X] : bigp(X) )).
fof(pel244a,axiom, ( ! [X] : ( bigq(X) =¿ bigs(X) ) )).
%----------------------------------------------------------------------
```

Figure 3: The presentation of Pelletier's problem 24a in TPTP syntax, called SYN054a.

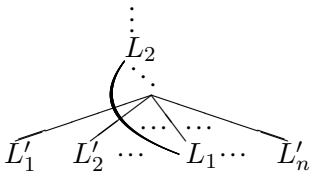
Table 1: The rules of the connection calculus using the connection tableau representation.

Start



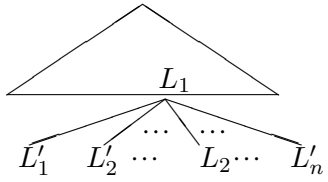
C_2={$L_1, \ldots, L_n$} and
C_2 is copy of $C_1 \in M$

Reduction



L_2 is literal on path and
{L_1, L_2} is σ-complementary

Extension



L_1 is leaf, C_2={$L'_1, \ldots, L_2, \ldots, L'_n$}
and C_2 is copy of $C_1 \in M$, $L_2 \in C_2$,
{L_1, L_2} is σ-complementary

`file(`*file_name*`,`*file_info*`)`
`inference(`*inference_name*`,`*inference_info*`,`*parents*`)`

The former is used for formulae taken from the problem file. The latter is used for inferred formulae, in which *inference_name* is the name of the inference rule, *inference_info* is a list of additional information about the inference, and *parents* is a list of the parents' node names in the derivation. The *inference_info* contains items such as variable bindings captured within `bind`/2 terms.

3.2 The leanCoP Syntax

According to [20], a refutation (or proof) is a derivation that has the root node *false*, representing the empty clause. Whereas this description fits proofs in resolution calculi, it is not directly applicable to proofs in sequent, tableau or connection calculi. An attempt to "squeeze" connection proofs into this resolution-style frame [15] resulted in a syntax that does not conform to the first two requirements suggested in Section 2.1: readability and simplicity. For this reason, leanCoP [14, 11] uses a syntax, called *leanTPTP*, that represents connection proofs in a more natural way, while still using main components of the TPTP/TSTP syntax.

In order to shorten the output, leanCoP returns connection proofs in the connection tableau representation [8]. The rules of the *connection tableau calculus* are shown in Table 1 and correspond to the rules of the formal calculus in Figure 1. There is no explicit *Axiom*, which is applied exactly once for each *Start* or *Extension* step in the formal calculus. Instead, a derivation is a proof if each literal in a leaf is included in a σ-complementary connection {L_1, L_2}.

Example 3 *A proof of Pelletier's problem 24a from Example 1 using the connection tableau representation is depicted in Figure 4. The rightmost literal in each proof step (number in parentheses) is annotated with the used clause number and (possibly) the term substitution that is applied to the clause. Proof step 1 is a Start step, step 7 is a Reduction step, all others are Extension steps.*

A connection proof using the leanTPTP syntax is a list of proof steps of the form

`cnf(`*step_name*`,plain,`*clause*`,`*rule_info*`).`

in which *step_name* is the name of the proof step, *clause* is the used (instantiated) clause C_2, and *rule_info* has one of the following forms:

`start(`*clause_number*`,` *term_substitution*`)`
`reduction(`*step_name*`)`
`extension(`*clause_number*`,` *term_substitution*`)`

step_name is the name of the proof step and of the form '$N_0.N_1....N_n$', in which the N_n-th "open" literal L_1 of its parent clause '$N_0.N_1....N_{n-1}$' is used in a Reduction or Extension step; the name of the Start step 'N_0' is '1'. For example, '1.2.2' is the proof step from the second open literal of its parent clause in the proof step named '1.2'. *step_name* in Reduction steps refer to the proof step with the clause that contains the literal L_2. *clause_number* is the number of the used *clause* in the matrix. *term_substitution* is of the form `bind([`*vlist*`,`*tlist*`])`, in which *vlist* is a list of variables $V_1, \ldots, V_n$ and *tlist* is a list of terms $t_1, \ldots, t_n$, such that $\sigma(V_i) = t_i$.

Example 4 *The proof of Pelletier's problem 24a from Example 1 using the leanTPTP syntax is shown in Figure 5. It is produced by* leanCoP *2.2 [14, 11] on the input file in Figure 3 (with simplified variable names and literals L_2 of Extension steps moved to the beginning of clauses).*

3.3 The Connect++ Syntax

Connect++ [6] is a C++ implementation of the connection calculus shown in Figure 1. The proof format of the Connect++ prover uses a stack to represent connection proofs. Each element on the stack corresponds to exactly one proof step in the (formal) connection calculus, and the stack is ordered to represent a depth-first search from the Start rule, exploring the left subtrees of Extensions first. Clauses, and literals within clauses, are numbered from 0. The first element of a proof is $\texttt{start}(i)$, giving the index of the start clause used. Reductions are represented as $\texttt{reduction}(p, q)$ where p denotes the index of the literal L_1 in $C \cup \{L_1\}$ and q denotes the index of the element in the path corresponding to L_2. Extensions are represented by $\texttt{left_branch}(p, i, j, d)$ and $\texttt{right_branch}(d)$ where p again denotes the index of the literal L_1 in $C \cup \{L_1\}$, i is the index of C_2, j the index of L_2 and d the proof tree depth.

One aim of the Connect++ syntax is to be quite explicit in stating how the rules of the connection calculus have been applied. This in turn stems from the idea that the simpler a proof checking program is, the more trustworthy it is likely to be, and this simplicity is supported by giving it as much information as possible. In the case of leanCoP, for example, a simplified version might be appropriate because L_1 is always the literal at index 0; however this might not be the case, for example, if a prover includes a heuristic for choosing an arbitrary literal with the aim of finding a proof more efficiently.

Example 5 *The proof of Pelletier's problem 24a from Example 1 using the Connect++ syntax is shown*

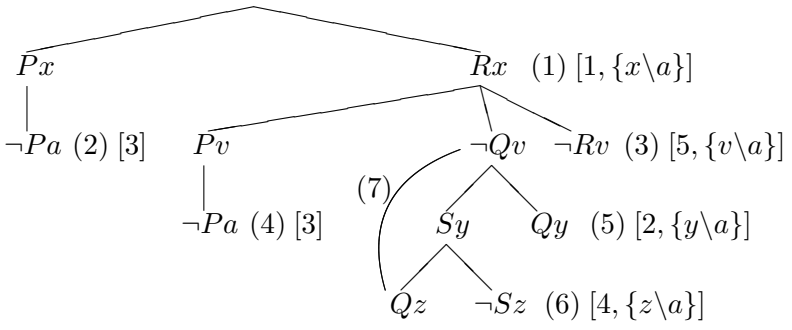


Figure 4: A proof for M_a using the connection tableau representation.

```
%--------------------------------------------------
cnf(1, plain, [bigp(X), bigr(X)], clausify(pel24)).
cnf(2, plain, [bigs(X), bigq(X)], clausify(pel241)).
cnf(3, plain, [-(bigp(a))], clausify(pel243a)).
cnf(4, plain, [bigq(X), -(bigs(X))], clausify(pel244a)).
cnf(5, plain, [bigp(X), -(bigq(X)), -(bigr(X))], clausify(pel242)).

cnf('1',plain,[bigp(a),bigr(a)],start(1,bind([[X],[a]]))).
cnf('1.1',plain,[-(bigp(a))],extension(3)).
cnf('1.2',plain,[-(bigr(a)),bigp(a),-(bigq(a))],extension(5,bind([[X],[a]]))).
cnf('1.2.1',plain,[-(bigp(a))],extension(3)).
cnf('1.2.2',plain,[bigq(a),bigs(a)],extension(2,bind([[X],[a]]))).
cnf('1.2.2.1',plain,[-(bigs(a)),bigq(a)],extension(4,bind([[X],[a]]))).
cnf('1.2.2.1.1',plain,[-(bigq(a))],reduction('1.2')).
%--------------------------------------------------
```

Figure 5: A proof of Pelletier's problem 24a using the leanTPTP syntax as returned by leanCoP 2.2.

```
matrix(0, [ -bigp(X), -bigr(X) ]).
matrix(1, [ -bigs(X), -bigq(X) ]).
matrix(2, [ bigp(a) ]).
matrix(3, [ -bigq(X), bigs(X) ]).
matrix(4, [ -bigp(X), bigq(X), bigr(X) ]).
proofstack([
start(0),
leftbranch(0, 2, 0, 2), rightbranch(2), leftbranch(0, 4, 2, 3),
leftbranch(0, 2, 0, 4), rightbranch(4), leftbranch(0, 3, 0, 5),
leftbranch(0, 1, 0, 6), reduction(0, 1), rightbranch(6),
rightbranch(5), rightbranch(3)
]).
```

Figure 6: A proof of Pelletier's problem 24a using the syntax as returned by CONNECT++.

in Figure 6.[5] *In this proof,* `start(0)` *denotes that clause* 0 *is used by the Start rule, and* `left_branch(0,2,0,2)` *denotes that the first Extension uses the first literal in* $C \cup \{L_1\}$*, clause* 2 *and literal* 0 *within that clause, and is at depth* 2 *in the tree, and so on.*

CONNECT++includes a proof checker. It is a short piece of Prolog code and uses Prolog's unification to build the substitution. However, it is straightforward to extend CONNECT++'s syntax to specify explicitly the substitution of the returned connection proof.

4 A Proposed Syntax for Connection Proofs

We propose a syntax, which follows more closely the TSTP format described in Section 3.1 and generalizes the leanTPTP format presented in Section 3.2 with the following modifications:

- include information about the *parent* node, which allows the use of arbitrary proof step *names* in the first argument,
- use the *rule_info* argument of the `inference` expression to specify clause name of C_1, position of literal L_1 in the parent clause, position of literal L_2 in C_2 (or in the active path), substitution σ applied to C_2 and the (instantiated) active *Path*.

[5] Note that the prover produces a slightly different proof of the problem by way of a different choice of C_2 in the third Extension. Also, literals are negated throughout the matrix as CONNECT++ works in the negative (conjunctive normal form) representation, starting with the negation of the stated problem.

```
%----------------------------------------------------
fof(pel241, axiom,  ?[X]:(bigs(X)&bigq(X)), file('SYN054a',pel241)).
fof(pel243a, axiom, ?[X]: bigp(X), file('SYN054a',pel243a)).
fof(pel244a, axiom, ![X]:(bigq(X)=¿bigs(X)), file('SYN054a',pel244a)).
fof(pel242, axiom, ![X]:(bigp(X)=¿bigq(X)—bigr(X)), file('SYN054a',pel242)).
fof(pel24, conjecture, ?[X]:(bigp(X) & bigr(X)), file('SYN054a',pel24)).

cnf(1, plain, [bigp(X), bigr(X)], clausify(pel24)).
cnf(2, plain, [bigs(X), bigq(X)], clausify(pel241)).
cnf(3, plain, [-(bigp(a))], clausify(pel243a)).
cnf(4, plain, [bigq(X), -(bigs(X))], clausify(pel244a)).
cnf(5, plain, [bigp(X), -(bigq(X)), -(bigr(X))], clausify(pel242)).

cnf(6,plain,[bigp(a),bigr(a)],inference(start,[1,bind([X],[a]),path([])],[])).
cnf(7,plain,[-(bigp(a))],inference(extension,[3,lit(1,1),path([bigp(a)])],[6])).
cnf(8,plain,[bigp(a),-(bigq(a)),-(bigr(a))],
      inference(extension,[5,lit(2,3),bind([[X],[a]]),path([bigr(a)])],[6])).
cnf(9,plain,[-(bigp(a))],inference(extension,[3,lit(1,1),path([bigr(a),bigp(a)])],[8])).
cnf(10,plain,[bigs(a),bigq(a)],
       inference(extension,[2,lit(2,2),bind([[X],[a]]),path([bigr(a),-(bigq(a))])],[8])).
cnf(11,plain,[bigq(a),-(bigs(a))],
    inference(extension,[4,lit(1,2),bind([[X],[a]]),path([bigr(a),-bigq(a)),bigs(a)])],[10])).
cnf(12,plain,[-(bigq(a))],
       inference(reduction,[lit(1,2),path([bigr(a),-(bigq(a)),bigs(a)])],[11])).
%----------------------------------------------------
```

Figure 7: A connection proof of Pelletier's problem 24a using the proposed syntax.

A (clausal) connection proof using the proposed syntax is a list of proof steps of the form

`cnf(`*name*`,plain,` *formula*`,inference(`*rule_name*`,` *rule_info*`,` *parent*`)).`

in which *name* is the name of the proof step, *formula* is the used (instantiated) clause C_2 (in case of a Start or Extension step) or a list containing L_2 (in case of a Reduction step),[6] *rule_name* is one of `start`, `reduction` or `extension`, and *parent* is a list containing the name of the previous (parent) proof step; furthermore, *rule_info* is a list of the form

`[`*clause_name*`,lit(`*lit1,lit2*`),bind([`*vlist*`,` *tlist*`]),path(`*plist*`)]`

in which *clause_name* is the name of the (input) clause C_1, *lit1* is the position of the literal L_1 in the parent clause (in case of a Reduction or Extension step), *lit2* is the position of the literal L_2 in the clause C_2 (in case of an Extension step) or in the active path (in case of a Reduction step), *vlist* is a list of variables $V_1, ..., V_n$ in C_2 and *tlist* is a list of terms $t_1, ..., t_n$, such that $\sigma(V_i) = t_i$, *plist* is the list of literals in the *active path*.[7] For the Start step, *parent* is empty, for Reduction steps, the argument *clause_name* is omitted. The arguments *lit*, *bind* and *path* can be omitted.[8]

Example 6 *The proof of Pelletier's problem 24a from Example 1 using the described syntax is shown in Figure 7.*[9] *A simplified proof omitting redundant arguments is shown in Figure 8.*

[6] Clauses C_1, C_2 and literals L_1, L_2 refer to the corresponding elements of the connection calculus in Figure 1.

[7] In the future, TSTP *inference status information* might be included. Note the fact that once a proof is found, it is not relevant for its *presentation* how the (rigid) substitution σ was calculated (for example, by using *free variables*).

[8] These arguments can be re-calculated from the other (given) information or might not be relevant for some users (for example, the order of branches in the proof tree).

[9] The presentation of the input clauses (proof steps pel24* and proof steps 1 to 5) follows the standard TPTP syntax.

```
%--------------------------------------------------------
cnf(1, plain, [bigp(X), bigr(X)], clausify(pel24)).
cnf(2, plain, [bigs(X), bigq(X)], clausify(pel241)).
cnf(3, plain, [-(bigp(a))], clausify(pel243a)).
cnf(4, plain, [bigq(X), -(bigs(X))], clausify(pel244a)).
cnf(5, plain, [bigp(X), -(bigq(X)), -(bigr(X))], clausify(pel242)).

cnf(6,plain,[bigp(a),bigr(a)],inference(start,[1],[])).
cnf(7,plain,[-(bigp(a))],inference(extension,[3],[6])).
cnf(8,plain,[bigp(a),-(bigq(a)),-(bigr(a))],inference(extension,[5],[6])).
cnf(9,plain,[-(bigp(a))],inference(extension,[3],[8])).
cnf(10,plain,[bigs(a),bigq(a)],inference(extension,[2],[8])).
cnf(11,plain,[bigq(a),-(bigs(a))],inference(extension,[4],[10])).
cnf(12,plain,[-(bigq(a))],inference(reduction,[],[11])).
%--------------------------------------------------------
```

Figure 8: A simplified connection proof of Pelletier's problem 24a using the proposed syntax.

5 Conclusion

In this paper, we described the syntax of the connection proofs returned by two existing connection provers for classical first-order logic, and proposed a syntax for a proof format that fulfils the requirements listed in Section 2.1. As mentioned in the introduction, it should not be seen as a final specification, but as a foundation to initiate discussions within the community in order to finalize such a specification taking further proof formats [4, 7, 18, 21] into account.

Compared to the (previous) leanTPTP syntax, the proposed syntax follows more closely the TPTP/TSTP syntax, retaining the slightly simpler representation of proofs as connection tableau, while also including additional information, such as the active path. The latter information is useful when proof steps are to be carried out independently, e.g., in parallel, as all necessary information is included in the description of each proof step. Similar to the TPTP/TSTP format, the leanTPTP format and the Connect++ format, the proposed proof format uses Prolog terms to represent connection proofs. This allows the use of simple Prolog code to implement proof checkers as done, for example, in leanCoP and Connect++.

A common standard for presentation of derivations and proofs will increase the interoperability between ATP systems, ATP tools, and application software. For example, Connect++ includes a tool to translate connection proofs into a LaTeX output of the formal calculus. A standardized syntax would make it possible to use this tool in combination with other connection provers. Such a standard would also allow the development of tools to translate connection proofs into sequent proofs [5], which are often used in interactive proof editors, such as Coq [1], NuPRL [3] or PVS [16]. To this end, a standardized syntax for sequent proofs would be desirable as well.

A possible extension of the current work includes the specification of a syntax to represent non-clausal connection proofs [12] or connection proofs in non-classical logics [10, 13]. These extensions should be kept in mind already when developing a format for clausal connection proofs in classical logics. For example, the connection calculi for intuitionistic and modal logic use an additional prefix substitution, which could be just added to the *rule_info* argument. Furthermore, future work include the specification of proof formats for other popular proof calculi, such as sequent calculi or (standard) tableau calculi.

References

[1] Y. Bertot and P. Casteran. *Interactive Theorem Proving and Program Development - Coq'Art*. Texts in Theoretical Computer Science. Springer-Verlag, 2004.

[2] W. Bibel. *Automated Theorem Proving*. Vieweg and Sohn, 1987.

[3] R. Constable, et al. *Implementing Mathematics with the Nuprl Proof Development System*. Prentice-Hall, 1986.

[4] M. Färber, C. Kaliszyk, and J. Urban. Machine Learning Guidance and Proof Certification for Connection Tableaux. arXiv:1805.03107v3 [cs.LO], 2018.

[5] G. Gentzen. Untersuchungen über das logische Schließen. *Mathematische Zeitschrift*, 36:176–210, 405–431, 1935.

[6] S. B. Holden. Connect++: A New Automated Theorem Prover Based on the Connection Calculus. In J. Otten and W. Bibel, editors, *Workshop on Automated Reasoning with Connection Calculi (AReCCa)*, CEUR Workshop Proceedings. CEUR-WS.org, 2023.

[7] C. Kaliszyk, J. Urban, and J. Vyskočil. Certified Connection Tableaux Proofs for HOL Light and TPTP. In *Certified Programs and Proofs (CPP)*, pages 59–66, ACM, 2015.

[8] R. Letz and G. Stenz. Model Elimination and Connection Tableau Procedures. In A. Robinson and A. Voronkov, editors, *Handbook of Automated Reasoning*, pages 2015–2114. Elsevier Science, 2001.

[9] D.W. Loveland. Mechanical Theorem Proving by Model Elimination. *Journal of the ACM*, 15(2):236–251, 1968.

[10] J. Otten. Clausal Connection-Based Theorem Proving in Intuitionistic First-Order Logic. In B. Beckert, editor, *TABLEAUX 2005*, LNAI, volume 3702, pages 245–261. Springer, 2005.

[11] J. Otten. Restricting Backtracking in Connection Calculi. *AI Communications*, 23(2-3):159–182, 2010.

[12] J. Otten. A Non-clausal Connection Calculus. In K. Brünnler, G. Metcalfe, editors, *TABLEAUX 2011*, LNAI, volume 6793, pages 226–241. Springer, 2011.

[13] J. Otten. MleanCoP: A Connection Prover for First-Order Modal Logic. In S. Demri, D. Kapur, C. Weidenbach, editors, *IJCAR 2014*, LNAI, volume 8562, pages 269–276. Springer, 2014.

[14] J. Otten and W. Bibel. leanCoP: Lean Connection-Based Theorem Proving. *Journal of Symbolic Computation*, 36(1-2):139–161, 2003.

[15] J. Otten and G. Sutcliffe. Using the TPTP Language for Representing Derivations in Tableau and Connection Calculi. In B. Konev, R.A. Schmidt, S. Schulz, editors, *PAAR 2010*, EPiC, volume 9, pages 95–105. EasyChair, 2010.

[16] S. Owre, S. Rajan, J.M. Rushby, N. Shankar, and M. Srivas. PVS: Combining Specification, Proof Checking, and Model Checking. In R. Alur and T.A. Henzinger, editors, *Computer-Aided Verification*, LNCS, volume 1102, pages 411–414. Springer, 1996.

[17] F.J. Pelletier. Seventy-five Problems for Testing Automatic Theorem Provers. *Journal of Automated Reasoning*, 2(2):191–216, 1986.

[18] G. Reis. Importing SMT and connection proofs as expansion trees. *Electronic Proceedings in Theoretical Computer Science*, 186:3–10, 2015.

[19] G. Sutcliffe. The TPTP problem library and associated infrastructure. *Journal of Automated Reasoning*, 59(4):483–502, 2017.

[20] G. Sutcliffe, S. Schulz, K. Claessen, and A. Van Gelder. Using the TPTP Language for Writing Derivations and Finite Interpretations. In U. Furbach and N. Shankar, editors, *IJCAR 2006*, LNAI, volume 4130, pages 67–81. Springer, 2006.

[21] A. Wolf and J. Schumann. ILF-SETHEO. In W. McCune, editor, *CADE-14*, LNCS, volume 1249, pages 61–64. Springer, 1997.